데이터 시각화 프레임워크:
Streamlit

Copyright©2025 By Donghyun Lee. All right reserved.

목차

✓1. Streamlit 소개

✓2. 설치 및 실행

✓3. 데모 페이지 탐색

✓4. Streamlit 구성 요소

✓5. 실습

✓6. 핵심 정리

✓7. 참고자료

1. Streamlit 소개

✓Streamlit 이란?



- ✓Python 기반의 데이터 웹 애플리케이션 프레임워크
- ✓복잡한 웹 기술 없이 간단한 코드로 웹 대시보드 제작 가능
- ✓데이터 분석 결과 시각화에 특화되어, 웹 형태로 쉽게 표현 및 공유 가능
- ✓오픈소스로 제공되며, 데이터 과학 및 AI 분야에서 널리 사용됨

1. Streamlit 소개

- ✓Streamlit 의 필요성
  - ✓데이터 분석 결과의 효과적인 시각화 필요
  - ✓웹을 통한 효율적이고 상호작용 가능한 전달 방식
  - ✓Python 코드만으로 웹 대시보드의 표현 가능
  - ✓추가적인 데이터 처리 없이 빠르고 간단하게 결과 공유 가능

1. Streamlit 소개

- ✓ Streamlit의 주요 기능
  - ✓쉬운 API
    - ✓st.write(),st.dataframe(),st.line_chart() 등 쉽게 활용 가능
  - ✓Interactive 위젯
    - ✓체크박스, 버튼, 선택상자 등 사용자가 상호작용 가능한 기능 제공
  - ✓다양한 시각화 요소
    - ✓선형그래프, 막대그래프, 지도 등 기본 차트 및 Plotly나 3D 시각화 연계 가능
  - ✓Streamlit Cloud 배포
    - ✓Github 연동으로 배포 및 버전 관리

2. 설치 및 실행

✓설치: 터미널 또는 명령 프롬프트에서 간단하게 설치

1. 작업 폴더 만들기
2. 가상환경 생성
3. Streamlit 설치

   pip install streamlit

4. 설치 확인

   streamlit -version

5. 데모 페이지 실행

   streamlit hello

2. 설치 및 실행

✓가상환경 생성 및 실행 (Windows / Mac)

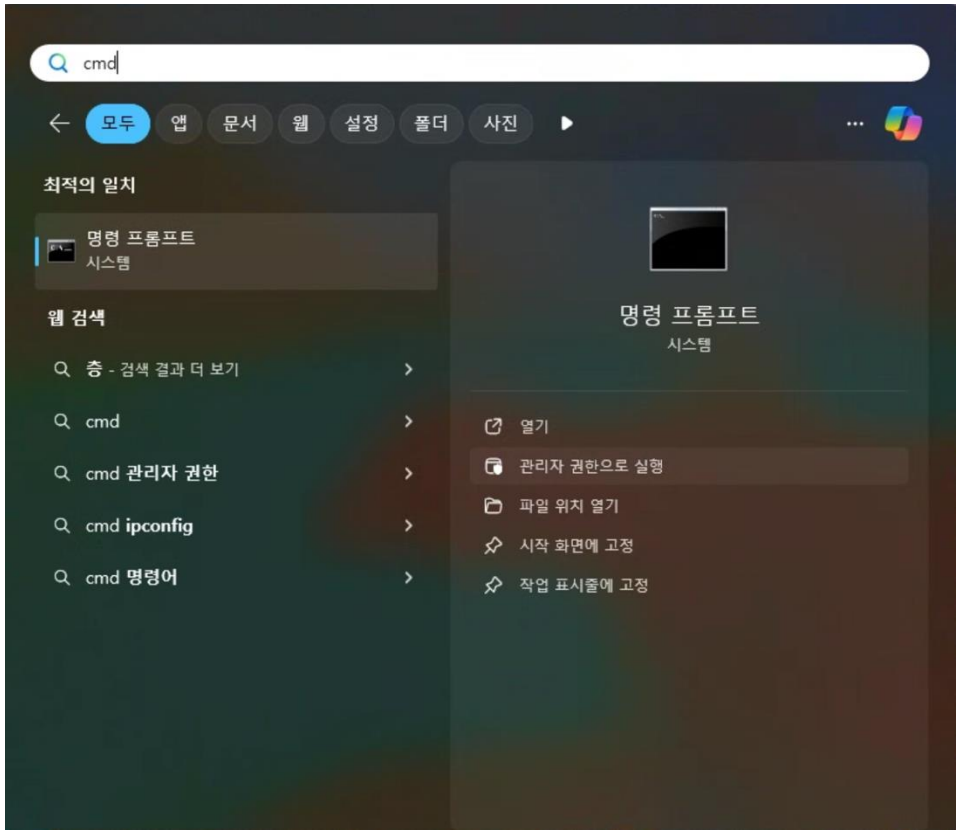
✓가상환경은 패키지 외부 패키지 설치에 영향을 받지 않는 환경으로, 모든 패키지를 새로 설치해야 함.

구분	Windows	macOS
폴더 이동	cd C:\streamlit_test	cd ~/streamlit_test
가상환경 생성	python -m venv venv	python3 -m venv venv
가상환경 활성화	venv\Scripts\activate	source venv/bin/activate
Python 실행	python	python3

2. 설치 및 실행

✓가상환경 생성 및 실행

1. 윈도우 검색창에서 명령 프롬프트 우클릭 -> 관리자 권한으로 실행

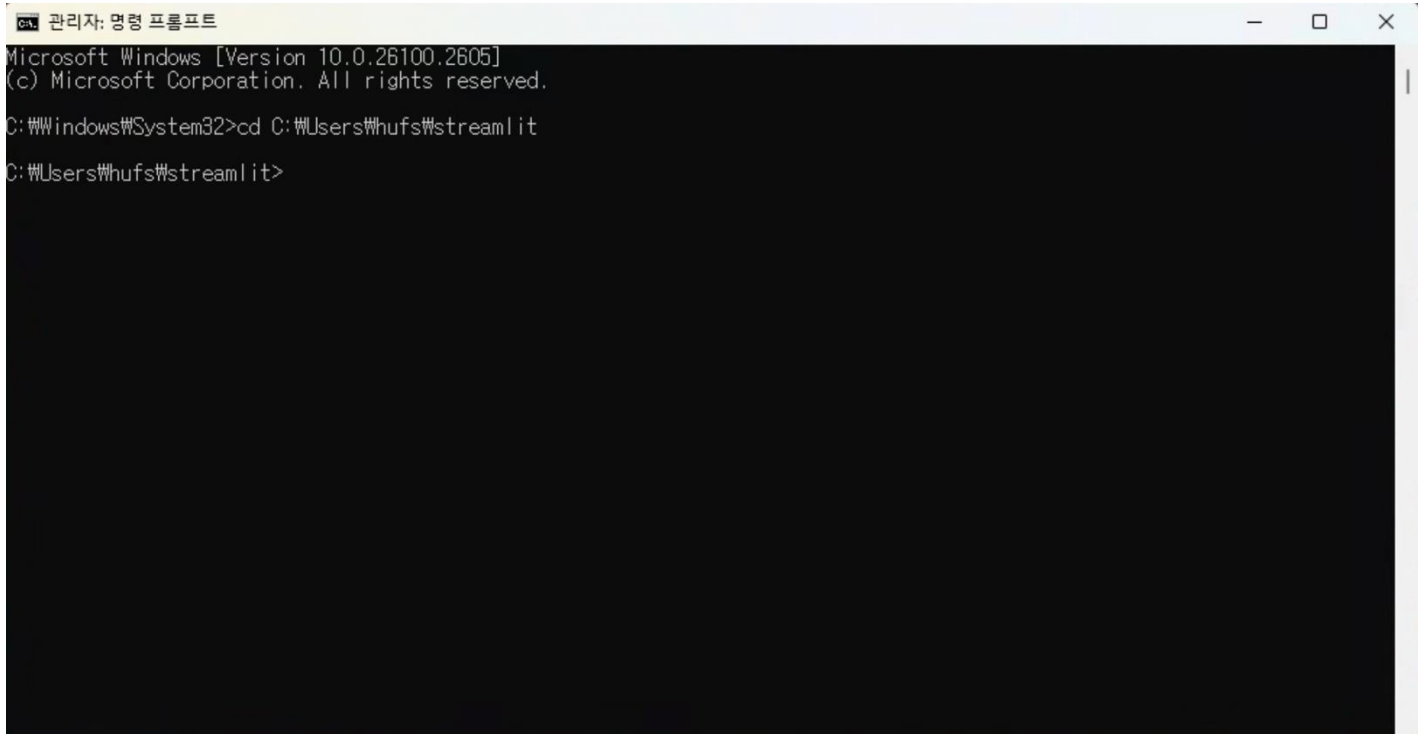


2. 설치 및 실행

✓가상환경 생성 및 실행

2. 파일을 만든 폴더에 진입

(ex cd C:\Users\hufs\streamlit)



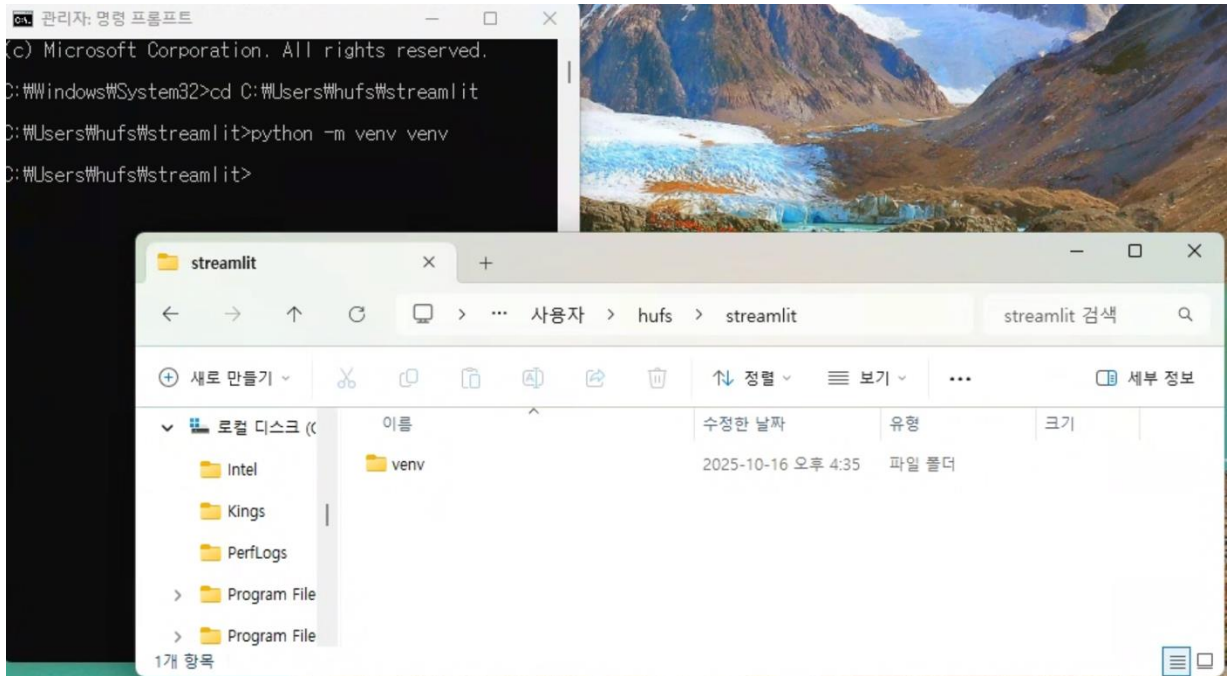
2. 설치 및 실행

✓가상환경 생성 및 실행

3. 가상환경 생성

python -m venv venv

해당 폴더 내부에 venv 폴더 생성



2. 설치 및 실행

✓가상환경 생성 및 실행

4. 가상환경 실행

venv\Scripts\activate 혹은

C:\Users\hufs\streamlit\venv\Scripts\activate

명령줄 앞에 (venv) 표시가 생기면 성공



2. 설치 및 실행

✓가상환경 생성 및 실행

4. 가상환경 실행 (MacOS)

source venv/bin/activate

명령줄 앞에 (venv) 표시가 생기면 성공



2. 설치 및 실행

✓Streamlit 설치 및 확인

✓pip install streamlit

```
(venv) C:₩Users₩hufs₩streamlit>pip install streamlit
Collecting streamlit
  Using cached streamlit-1.50.0-py3-none-any.whl.metadata (9.5 kB)
Collecting altair!=5.4.0,!=5.4.1,<6,>=4.0 (from streamlit)
  Using cached altair-5.5.0-py3-none-any.whl.metadata (11 kB)
Collecting blinker<2,>=1.5.0 (from streamlit)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Successfully installed MarkupSafe-3.0.3 altair-5.5.0 attrs-25.4.0 blinker-1.9.0 cachetools-6.2.
rmalizer-3.4.4 click-8.3.0 colorama-0.4.6 gitdb-4.0.12 gitpython-3.1.45 idna-3.11 jinja2-3.1.6
pecifications-2025.9.1 narwhals-2.8.0 numpy-2.3.4 packaging-25.0 pandas-2.3.3 pillow-11.3.0 pro
deck-0.9.1 python-dateutil-2.9.0.post0 pytz-2025.2 referencing-0.37.0 requests-2.32.5 rpds-py-0
treamlit-1.50.0 tenacity-9.1.2 toml-0.10.2 tornado-6.5.2 typing-extensions-4.15.0 tzdata-2025.2
```

✓streamlit --version (‘-’ 2개)

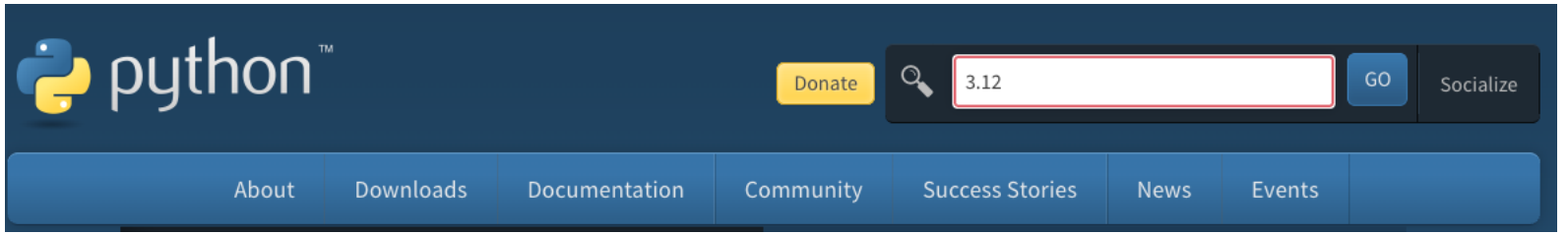
```
(venv) C:₩Users₩hufs₩streamlit>streamlit --version
Streamlit, version 1.50.0

(venv) C:₩Users₩hufs₩streamlit>
```

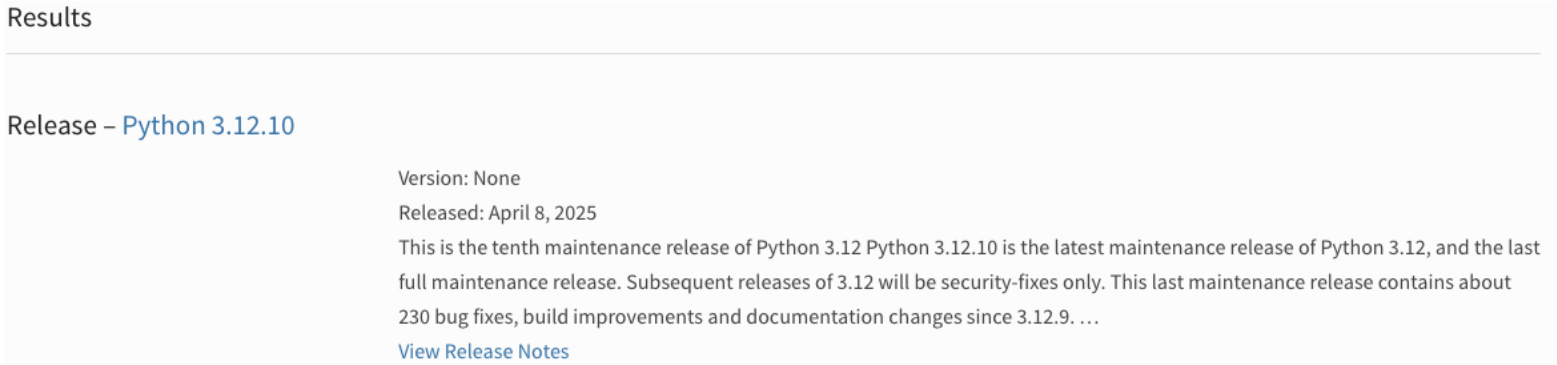
2. 설치 및 실행

✓Python 설치가 안되어 있을 때

✓https://www.python.org/ 접속 후, 3.12 검색



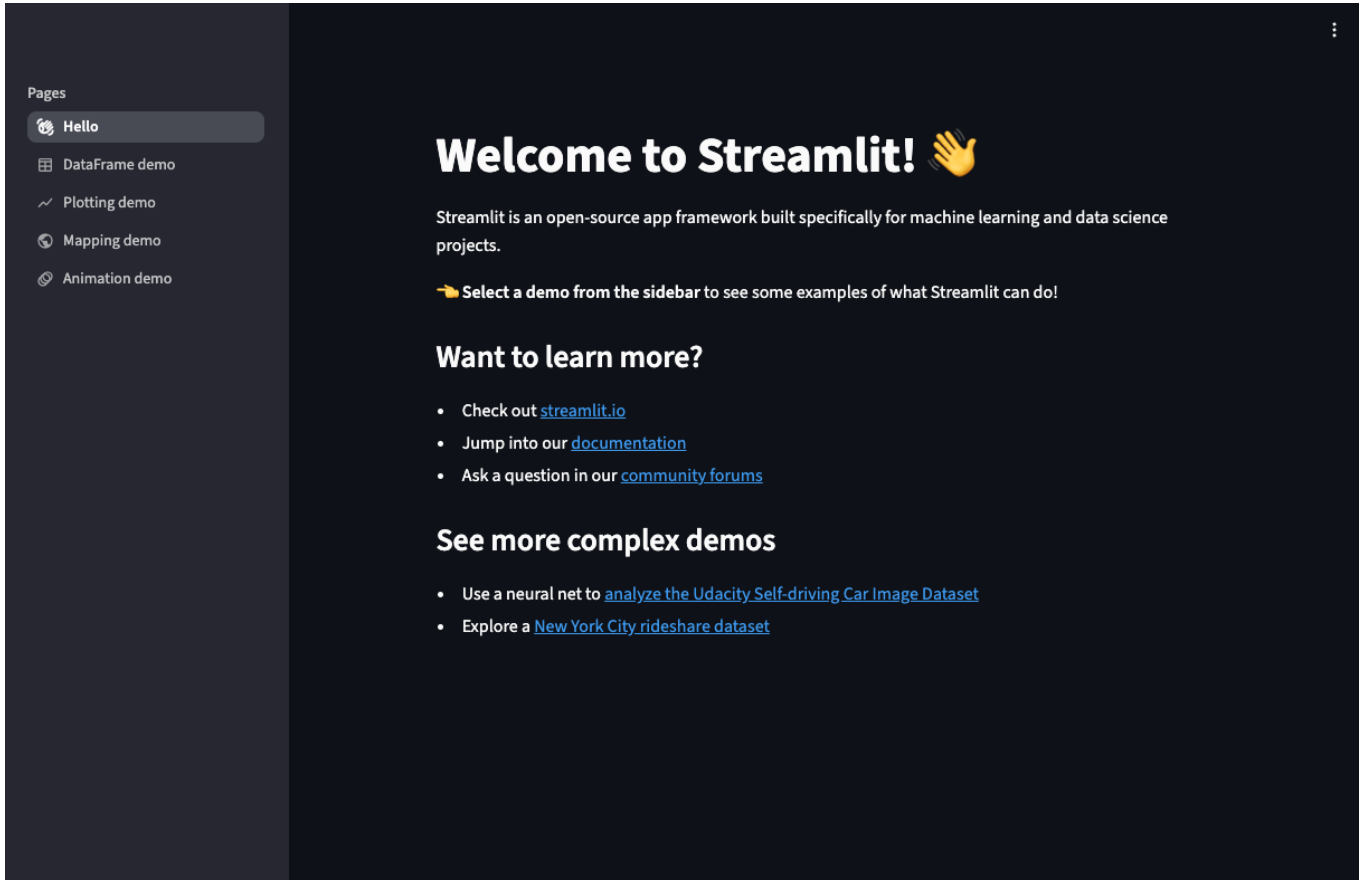
✓3.12.10 버전 선택 후, OS에 맞는 버전 다운로드



3. 데모 페이지 탐색

✓명령어: streamlit hello

✓접속 링크: localhost:8501



3. 데모 페이지 탐색

- ✓Streamlit 데모 페이지
  - ✓Hello: 기본 페이지
  - ✓Dataframe_demo: 표 및 기본 그래프
  - ✓Plotting_demo: 선형 그래프 및 시각화
  - ✓Mapping_demo: 좌표를 통한 3D 지도 표현
  - ✓Animation_demo: 프레임 단위로 움직이는 효과

3. 데모 페이지 탐색

- ✓DataFrame demo
  - ✓st.dataframe()을 이용해 DataFrame 시각화
  - ✓ 데이터 불러오기와 데이터 시각화
  - ✓상호작용에 따른 시각 데이터 변화

3. 데모 페이지 탐색

- ✓Plotting demo
  - ✓st.line_chart()로 실시간 그래프 차트 업데이트
  - ✓5개의 랜덤 데이터를 100회 생성하여 선형 그래프 생성

3. 데모 페이지 탐색

- ✓Mapping demo
  - ✓st.pydeck_chart()를 통해 여러 위치 데이터를 3D 지도 위에 시각화
  - ✓데이터 별로 다양한 레이어를 활용하여 시각화
  - ✓체크박스를 통해 레이어 선택 및 해제 가능

3. 데모 페이지 탐색

- ✓Animation demo
  - ✓프랙탈 패턴을 계산하여 실시간으로 이미지가 변화하는 애니메이션 시각화
  - ✓상호작용에 따라 계산 수준 조절
  - ✓프레임 단위로 이미지 생성 및 시각화를 반복

4. Streamlit 앱 구조

- ✓Streamlit 앱 구조
  - ✓Python 파일을 실행해 웹 페이지를 만드는 방식
  - ✓터미널에서 “streamlit run app.py” 명령으로 실행
  - ✓실행하면 브라우저가 자동으로 열리고 결과가 표시
  - ✓코드가 위에서 아래로 순차적으로 실행
  - ✓화면 조작 시 전체 코드가 다시 실행되어 화면이 갱신

4. Streamlit 구성 요소

- ✓Streamlit 기본 구성 요소
  - ✓제목 - Title
    - ✓st.title('This is the title ')
  - ✓헤더 - Header
    - ✓st.header('This is the header')
  - ✓서브헤더 - Subheader
    - ✓st.subheader('This is the subheader')
  - ✓텍스트 - Text
    - ✓st.text('This is the text')
  - ✓캡션 - Caption
    - ✓st.caption('This is the caption')

4. Streamlit 구성 요소

```
import streamlit as st

st.title("Streamlit 예시 페이지")
st.header("이건 Header 입니다")
st.subheader("이건 Subheader 입니다")

st.text("단순한 텍스트입니다.")
st.caption("이건 캡션(작은 글씨)입니다.")

st.markdown("**마크다운** 문법도 _사용_ 가능")
```

4. Streamlit 구성 요소

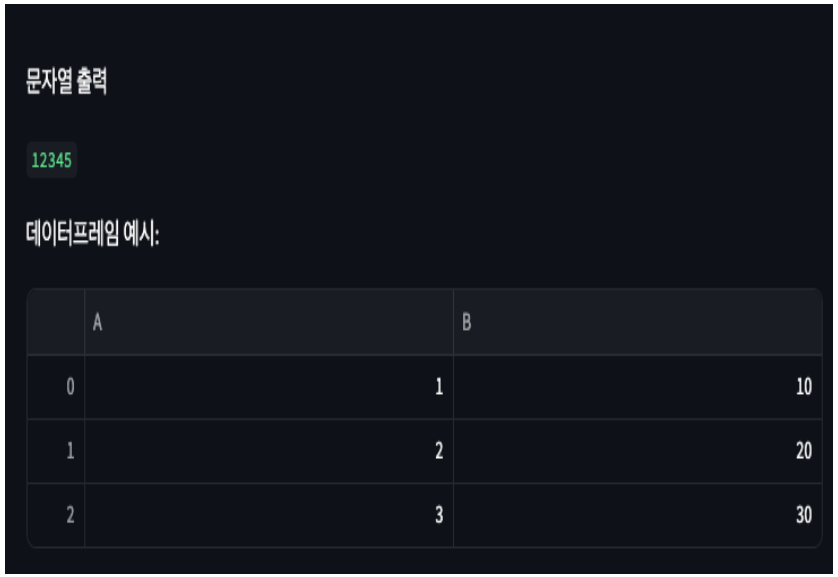
✓st.write()

✓범용적으로 가장 많이 쓰이는 출력 함수

✓문자열, 숫자, DateFrame 등 자동 인식하여 출력

✓st.write("문자열 출력")

```
st.write(12345)
df = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [10, 20, 30]
})
st.write("데이터프레임 예시:", df)
```



4. Streamlit 구성 요소

```
import streamlit as st
import pandas as pd

st.title("st.write() 예시")

st.write("이건 문자열 출력입니다.")
st.write("**마크다운 문법**도 자동 인식됩니다.")

st.write("계산 예시:", 10 + 5)
st.write("LaTeX 수식 예시:", r"$a^2 + b^2 = c^2$")

df = pd.DataFrame({
    "이름": ["A", "B", "C"],
    "점수": [85, 90, 78]
})
st.write("데이터프레임 예시:", df)
```

4. Streamlit 구성 요소

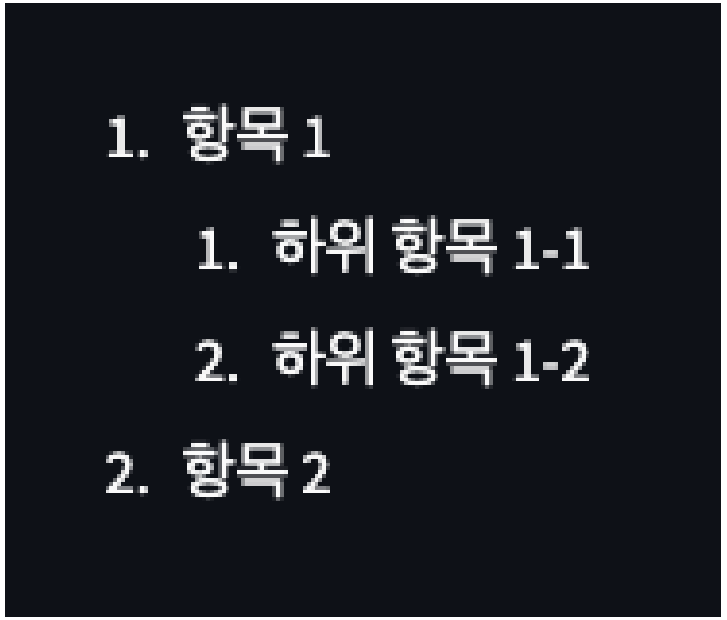
✓번호 리스트

✓숫자와 점으로 작성하며, 공백 3칸으로 단계 구분

✓st.markdown(

```
"1. 항목 1\n”
"   1. 하위 항목 1-1\n”
"   1. 하위 항목 1-2\n”
"1. 항목 2”
```

)



4. Streamlit 구성 요소

```
import streamlit as st

st.title("번호 리스트 예시")

st.markdown("""
1. 첫 번째 단계
2. 두 번째 단계
   1. 하위 단계 2-1
   2. 하위 단계 2-2
3. 세 번째 단계
""")
```

4. Streamlit 구성 요소

✓글머리 기호

✓- 또는 * 을 사용하며, 공백 2칸으로 하위 항목 표현

✓st.markdown(

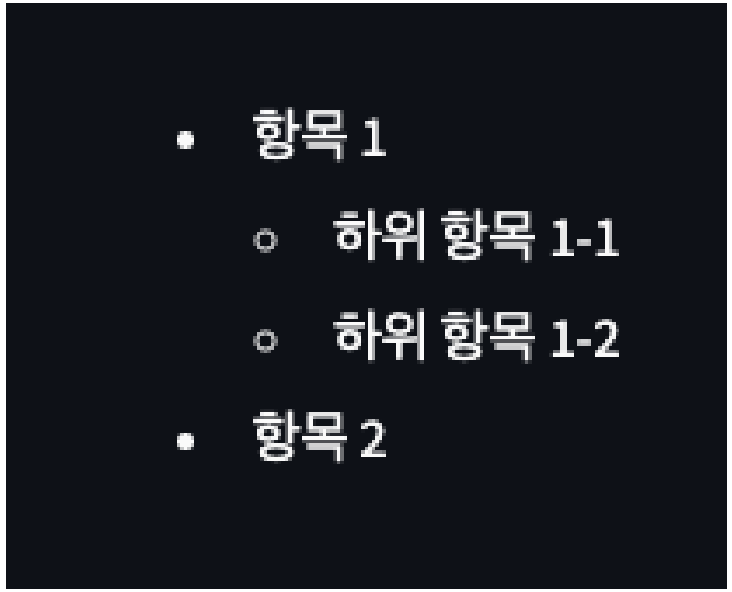
"- 항목 1\n”

" - 하위 항목 1-1\n”

" - 하위 항목 1-2\n”

"- 항목 2”

)



4. Streamlit 구성 요소

```
import streamlit as st

st.title("글머리 기호 예시")

st.markdown("""
- 첫 번째 항목
- 두 번째 항목
  - 하위 항목 1
  - 하위 항목 2
- 세 번째 항목
""")
```

4. Streamlit 구성 요소

- ✓기본 차트
  - ✓matplotlib 기반의 간단한 차트 함수를 제공
  - ✓st.line_chart(df) : 선형 그래프
  - ✓st.bar_chart(df) : 막대 그래프
  - ✓st.area_chart(df) : 면적 그래프

4. Streamlit 구성 요소

```
import streamlit as st
import matplotlib.pyplot as plt
import numpy as np

# 예시 데이터
x = np.linspace(0, 10, 50)
y1 = np.sin(x)
y2 = np.cos(x)

fig1, ax1 = plt.subplots()
ax1.plot(x, y1, label='sin(x)')
ax1.plot(x, y2, label='cos(x)')
ax1.legend()
st.pyplot(fig1)
```

```
fig2, ax2 = plt.subplots()
ax2.bar(["A", "B", "C"], [10, 25, 15])
st.pyplot(fig2)

fig3, ax3 = plt.subplots()
ax3.scatter(x, y1, color='red')
st.pyplot(fig3)
```

4. Streamlit 구성 요소

✓Altair 차트

✓데이터를 연결해 표현하는 시각화 라이브러리

✓st.altair_chart(df) 처럼 데이터 프레임 시각화

✓단, pivot 형태가 아닌 unpivot 형태를 사용해야 함

date	teamA	teamB
1/1	100	80
1/2	120	95
1/3	90	110

pivot

date	team	sales
1/1	A	100
1/1	B	80
1/2	A	120
1/2	B	95
1/3	A	90
1/3	B	110

unpivot

4. Streamlit 구성 요소

```
import altair as alt
import pandas as pd
import streamlit as st

df = pd.DataFrame({
    "date": ["1/1", "1/1", "1/2", "1/2", "1/3", "1/3"],
    "team": ["A", "B", "A", "B", "A", "B"],
    "sales": [100, 80, 120, 95, 90, 110]
})
chart = alt.Chart(df).mark_line().encode(
    x="date",
    y="sales",
    color="team"
)

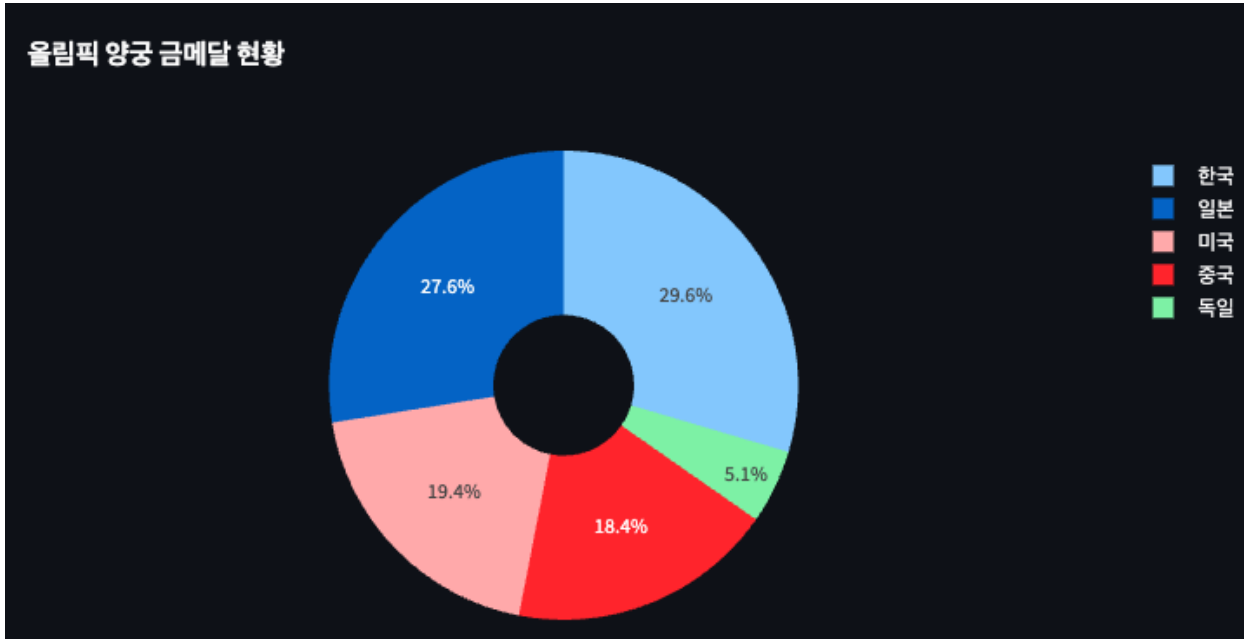
st.altair_chart(chart, use_container_width=True)
```

4. Streamlit 구성 요소

✓Plotly 차트

✓Plotly는 인터랙티브 시각화를 지원하는 라이브러리

✓마우스 오버, 확대/축소, 클릭 등 상호작용이 가능한 그래프 생성



4. Streamlit 구성 요소

```
import plotly.express as px
import streamlit as st
import pandas as pd

df = pd.DataFrame({
    "nation": ["한국", "미국", "일본", "중국", "독일"],
    "gold": [29, 19, 27, 18, 5],
    "silver": [3, 41, 14, 32, 11],
    "bronze": [9, 33, 17, 18, 16]
})

fig = px.pie(df, names='nation', values='gold',
             title='올림픽 양궁 금메달 현황', hole=.3)
st.plotly_chart(fig)
```

6. 정리

- ✓Streamlit 정리
  - ✓Python 기반의 데이터 시각화 웹 프레임워크
  - ✓웹 기술 없이 간단하게 상호작용 대시보드 제작 가능
  - ✓st.write, st.dataframe, st.line_chart, st.altair_chart, st.plotly_chart 등 다양하게 활용
  - ✓.py 파일을 저장 후 streamlit run 파일명.py 으로 실행
  - ✓Altair, Plotly 등 외부 라이브러리와 손쉽게 연동 가능

7. 참고 자료

구분	링크
공식 사이트	https://streamlit.io
공식 문서	https://docs.streamlit.io
GitHub 저장소	https://github.com/streamlit/streamlit
	https://velog.io/@sy508011/%EC%8A%A4%ED%8A%B8%EB%A6%BC%EB%A6%BF-1-%EC%84%A4%EC%B9%98-%EB%B0%8F-%EC%8B%A4%ED%96%89
	https://velog.io/@euisuk-chung/Streamlit-%EC%86%8C%EA%B0%9C-%EB%B0%8F-%ED%99%9C%EC%9A%A9-%EA%B0%80%EC%9D%B4%EB%93%9C

Q&A